HANDS-ON WORKSHOP

Building with open-source AI agents

From “what is an agent?” to the goose Development Kit: sessions, context engineering, recipes, tool permissions, MCP and ACP.

Azeez Syed · AIYatra

Khaja Moinuddin Mohammad · Organiser and founder, AIYatra

```
$ brew install block-goose-cli
$ goose configure
◇ Configure Providers
◇ Anthropic
└ Saved

$ goose session
 goose › _
```

Deconstruct the abstraction.

Expose the kernel truth.

COMMUNITY

Join AIYatra

An AI learning journey out of Manikonda, Hyderabad. Talks, workshops and hands-on sessions.

meetup.com/aiyatra

Scan to join

THE DAY, END TO END

Fourteen sections

01	What is an agent?
02	Introduction to goose
03	goose vs Claude Code, Cursor & Codex
04	Installation & configuration
05	The goose folder structure
06	Session management
07	Context engineering
08	Recipes & subrecipes
09	Managing tools & permissions
10	Security & safe autonomy
11	MCP surface & platform operations
12	The Agent Client Protocol
13	goose architecture
14	The goose Development Kit

Reference: goose-docs.ai/docs/category/guides

01 · WHAT IS AN AGENT?

Prompt, context, agent

Three layers, each larger than the last.

01

Prompt

The instruction, in natural language. On its own a model only produces text. No side effects, no state.

02

Context

Everything the model can see this turn: conversation history, file contents, tool schemas, project rules. Context quality sets the ceiling on output quality.

03

Agent

All three in a loop. The model proposes a tool call, the harness runs it, the result goes back. Repeat until the goal is met or a limit stops it.

Reference: goose-docs.ai/docs/goose-architecture

The harness decides what the agent can do

- The model is stateless and executes nothing. It emits a JSON tool call, and something else runs it.
- That something else decides what the model can touch, what happens when a call fails, and what stays in context.
- Swap the model and you change quality. Swap the harness and you change what the agent can do at all.
- goose, Claude Code, Cursor and Codex are all harnesses. They compete on loop design, tooling and safety.

Reference: goose-docs.ai/docs/goose-architecture

What it actually is

goose is a local agent runtime. It pairs an LLM with real tools and runs the loop on your machine: it reads and writes your files, runs shell commands, calls APIs, and keeps working until the task is done. It is not a chat window that suggests snippets.

It ships as a Desktop app and a CLI. Both share one configuration, and you can resume a session in either.

Open source
github.com/aaif-goose/goose · AAIF, under the Linux Foundation

Written in Rust
Local-first runtime; Desktop and CLI share ~/.config/goose

Provider-agnostic
Anthropic, OpenAI, Gemini, Ollama, OpenRouter, Tetrate and more

MCP-native
Extensions are Model Context Protocol servers

Reference: goose-docs.ai/docs/quickstart

What ships with it

- **Developer extension:** shell, file edit, and the analyze tool
- Memory, Computer Controller, Chat Recall and other bundled MCPs
- Persistent sessions in SQLite, resumable across CLI and Desktop
- Recipes: shareable, parameterised, one-click workflows
- Skills, subagents, hooks, plugins and custom slash commands
- Four permission modes, from full autonomy to read-only chat

```
$ goose session # interactive
$ goose run -i plan.md # headless
$ goose configure # providers
$ goose info -v # all settings
$ goose acp # ACP server
$ goose update # upgrade

goose › analyze the auth flow in src/
 and tell me what calls
 authenticate()
```

Reference: goose-docs.ai/docs/quickstart

— 03 · GOOSE VS CLAUDE CODE, CURSOR & CODEX

They are all harnesses

They differ in openness, form factor and who owns the runtime.

	goose	Claude Code	Cursor	Codex
Steward	AAIF / open source	Anthropic	Anysphere	OpenAI
Form factor	Desktop app + CLI + ACP server	CLI + IDE + desktop	Full IDE (VS Code fork) + CLI agent	CLI + cloud + IDE extension
Openness	Fully open source, Rust	Proprietary client	Proprietary	CLI open source, service is not
Model choice	Any provider; swap freely	Anthropic models	Mixed / routed models	OpenAI GPT-5 series
Extensibility	MCP extensions, recipes, skills, hooks, subagents	MCP servers, skills, subagents	IDE-centric rules and tooling	Skills, sandboxed execution
Where it wins	You own the runtime: embed or fork it	Deep Anthropic model integration	Best in-editor UX	Cloud-delegated long tasks

Reference: goose-docs.ai/docs/guides/acp-providers · /guides/cli-providers

03 · GOOSE VS CLAUDE CODE, CURSOR & CODEX

goose can drive the others

`claude-acp` `codex-acp`

`claude-code` `codex` `cursor-agent` `gemini-cli`

Configure `claude-acp` or `codex-acp` as ACP providers and goose delegates execution to Claude Code or Codex, while keeping its own sessions, recipes and scheduling.

The older pass-through CLI providers (`claude-code`, `codex`, `cursor-agent`, `gemini-cli`) still work but are deprecated, and they drop goose's extension ecosystem.

Reference: goose-docs.ai/docs/guides/acp-providers · /guides/cli-providers

04 · INSTALLATION & CONFIGURATION

Desktop, CLI, or both

They share one configuration store.

1 · INSTALL

```
# macOS / Linux - CLI
curl -fsSL https://github.com/aaif-goose/goose/\
  releases/download/stable/download_cli.sh | bash

# macOS via Homebrew
brew install block-goose-cli      # CLI
brew install --cask block-goose   # Desktop

# Windows (PowerShell)
.\download_cli.ps1

goose update                      # upgrade later
```

2 · SET AN LLM PROVIDER

```
$ goose configure

┌ goose-configure
◇ What would you like to configure?
│ Configure Providers
◇ Which model provider should we use?
│ Anthropic
◇ Provider requires ANTHROPIC_API_KEY
│ ••••••••••••••••••••
└ Configuration saved successfully
```

Reference: goose-docs.ai/docs/getting-started/installation

Four things to know before you start

Keys live in the keyring

Never in `config.yaml`; goose ignores keys there. Use the keyring, `secrets.yaml`, or an env var.

Desktop first-run options

Quick setup with an API key, ChatGPT subscription, Tetrate Agent Router, or OpenRouter.

Tool calling matters

goose leans on tool calling, and the docs say it works best with Claude 4 models.

Pin versions in CI

Set `GOOSE_VERSION` for reproducible, non-interactive installs in pipelines.

Run `goose info -v`. It prints every active setting and where it came from.

Reference: goose-docs.ai/docs/getting-started/installation

05 · THE GOOSE FOLDER STRUCTURE

Where state lives

```
~/.config/goose/                # macOS + Linux
├─ config.yaml                  provider, model,
│                               extensions, GOOSE_*
├─ permission.yaml              tool permission levels
├─ secrets.yaml                 keys, if no keyring
├─ permissions/
│  └─ tool_permissions.json     runtime decisions
└─ prompts/                     custom templates

~/.local/share/goose/
├─ sessions/sessions.db         SQLite (v1.10.0+)
└─ recipes/                     saved recipes
```

```
<your-repo>/
├─ AGENTS.md | .goosehints      project context
└─ .gooseignore                 paths goose must
                                not touch
```

PRECEDENCE: HIGHEST WINS

01 Environment variables

02 `config.yaml` settings

03 Built-in defaults

Windows: `%APPDATA%\Block\goose\config\config.yaml`

Reference: goose-docs.ai/docs/guides/config-files · /guides/environment-variables

Settings worth knowing on day one

`GOOSE_MODE`
auto | approve | smart_approve | chat

`GOOSE_MAX_TURNS`
Turns without input (default 1000)

`GOOSE_AUTO_COMPACT_THRESHOLD`
Default 0.8

`GOOSE_TEMPERATURE`, `GOOSE_MAX_TOKENS`
Sampling and output ceilings

`GOOSE_ALLOWLIST`
Restrict installable extensions

`SECURITY_PROMPT_ENABLED`
Injection detection

Reference: goose-docs.ai/docs/guides/environment-variables

05 · THE GOOSE FOLDER STRUCTURE

API keys in config.yaml are ignored

> silent failure
>
> goose does not read provider API keys from `config.yaml`. A key there is ignored, and fails later with “No api key passed in.” Store it in the system keyring with `goose configure`, or in `secrets.yaml` when there is no keyring, such as on headless servers, in containers and in CI. You can also export the provider’s environment variable, which wins over stored secrets.

Editing these files directly needs a restart before existing sessions see the change.

Reference: goose-docs.ai/docs/guides/config-files

06 · SESSION MANAGEMENT

A session is one continuous interaction

goose stores it and lets you resume it.

Start and name

goose names the session from your first prompt. IDs use YYYYMMDD_<count>. Set GOOSE_DISABLE_SESSION_NAMING in CI.

Resume

goose session -r --name <name>. The 10 most recent sit in the Desktop sidebar; older ones come from Session History.

Search

Cmd+F within a session, or across sessions from history. Or ask goose: the Chat Recall extension searches your history.

Duplicate vs fork

Duplicate copies a whole session to reuse its setup. Fork branches from a specific edited message to explore a different path.

Import and export

Export any session to JSON for backup, sharing or migration. Import creates a new session rather than overwriting.

Smart context

Automatic compaction at 80% of the window by default, plus a max-turns ceiling so a runaway loop stops on its own.

Reference: goose-docs.ai/docs/guides/sessions

06 · SESSION MANAGEMENT

Practitioner rules

- Sessions save on exit. Nothing to commit.
- Start in Desktop, resume in the CLI. One database, both interfaces.
- New task, new session. A stale one fills with contradictory context and the model starts fighting its own history.
- Delete in Desktop and it goes from the CLI too. No undo.

```
$ goose session
$ goose session list -l 1
 20260213_9 - react-migration
 2026-02-13 16:20 UTC

$ goose session -r --name react-migration

$ sqlite3 ~/.local/share/goose/\
 sessions/sessions.db \
 "SELECT id, description FROM sessions"
```

Reference: goose-docs.ai/docs/guides/sessions

— 07 · CONTEXT ENGINEERING

Define how you work once

Instead of re-explaining it every session. Twelve guides cover this area.

.goosehints
Project context, conventions and preferences. goose loads it at session start.

Agent Skills
Reusable instruction sets: workflows, scripts, resources. Loaded on demand, not always on.

Custom Agents
Create, edit, import and export specialised agents with reusable prompts and metadata.

Plugins
Installable packages that bundle skills, hooks and other reusable components.

Hooks
Run your own scripts on session start, prompt submit, tool call, file edit or shell execution.

Slash Commands
Custom /shortcuts that fire reusable instructions or launch a recipe in any chat.

Prompt Templates
Override the built-in prompts that govern how goose responds, plans and compacts.

Subagents
Delegate focused work to isolated instances, in sequence or in parallel, without polluting your context.

.gooseignore
Global or per-project rules for files and directories goose must never read or touch.

Planning Mode
Break complex work into reviewed steps before implementation starts. Supports a separate planner model.

Persistent Instructions
Goes into working memory every turn. For guardrails that must not drift.

Memory Extension
Durable knowledge across sessions: commands, snippets, preferences goose can recall later.

Reference: goose-docs.ai/docs/guides/context-engineering (12 guides)

07 · CONTEXT ENGINEERING

Pick the right tool

Hint

`.goosehints` always loads. Keep it small and stable.

Skill

Skills load on demand. Put long procedures there.

Memory

Memory persists across sessions. Use it for facts you will need again.

Persistent instruction

Re-injects every turn. Reserve it for hard guardrails.

Persistent instructions cost tokens on every call. Everything competes for the same context window, so budget it.

Reference: goose-docs.ai/docs/guides/context-engineering

— 08 · RECIPES & SUBRECIPES

Package a working session

Tools, prompt, parameters and settings, in one file a teammate launches in one click.

WHAT A RECIPE CARRIES

- The instructions and goal for the session
- Which extensions must be enabled
- Parameters the runner fills in at launch
- Provider, model and behaviour settings
- Optional subrecipes for delegated sub-tasks

SUBRECIPES

A recipe can call other recipes, in sequence or several in parallel. This is how you fan out repetitive work, such as auditing twenty services or migrating thirty files, without one context window holding all of it.

Reference: goose-docs.ai/docs/guides/recipes · goose-docs.ai/recipes

— 08 · RECIPES & SUBRECIPES

Bind one to a slash command

```
# config.yaml
slash_commands:
  - command: "run-tests"
    recipe_path: "/path/to/recipe.yaml"
  - command: "daily-standup"
    recipe_path: "~/.local/share/goose/\
                  recipes/standup.yaml"

# point goose at a shared team recipe repo
GOOSE_RECIPE_GITHUB_REPO:
  "aaif-goose/goose-recipes"
```

```
goose › /run-tests
```

IN THE DOCS

Reusable Recipes
Share a session setup others launch with one click, including via deeplink.

Recipe Reference
The full technical schema for authoring recipes from the CLI.

Saving Recipes
Save, organise and discover recipes; back them with a GitHub repo.

Recipe Cookbook
Ready-made recipes for common development scenarios, ready to adapt.

Reference: goose-docs.ai/docs/guides/recipes

09 · MANAGING TOOLS & PERMISSIONS

GOOSE_MODE sets four levels of autonomy

auto

goose runs tools without asking. Fast, and only for a sandbox or a throwaway branch.

smart_approve

goose judges risk and prompts only for consequential actions. The everyday default.

approve

Every tool call waits for your yes. Slow, but right when the target is production.

chat

No tools run. Conversation only, for design discussion and code review.

Reference: goose-docs.ai/docs/guides/managing-tools

09 · MANAGING TOOLS & PERMISSIONS

Underneath the global mode

Tool Permissions
Per-tool control layered under the global mode, stored in permission.yaml.

Code Mode
Rather than pre-loading every tool schema, goose discovers and calls MCP tools on demand.

Adjust Tool Output
Dial verbosity from full transcripts to clean summaries with GOOSE_CLI_MIN_PRIORITY.

Tool Filtering
available_tools in config.yaml loads only the tools you need. Fewer schemas, fewer tokens.

Ollama Tool Shim
Experimental interpreter layer that adds tool calling to local models without native support.

Extension Allowlist
GOOSE_ALLOWLIST restricts which MCP servers install. Needed in corporate estates.

Reference: goose-docs.ai/docs/guides/managing-tools

An agent with shell access is a supply-chain surface

Adversary Mode

A second agent watches tool calls as they run, so there is another opinion if the first agent goes somewhere it should not.
It is a runtime guard, not a policy file. It reads what runs, not what was declared up front.

Prompt Injection Detection

Screens for hijacking instructions before they run. The file you read, the issue you fetched and the web page you scraped are all untrusted input.
`SECURITY_PROMPT_ENABLED` turns it on; `SECURITY_PROMPT_THRESHOLD` (default 0.8) tunes strictness. You can self-host an ML classifier endpoint against a published API spec.

Four more controls

macOS Sandbox

Apple sandbox controls over file access, network connections and process restrictions for goose Desktop.

Extension Allowlist

Point GOOSE_ALLOWLIST at a URL of approved MCP servers so only vetted extensions install.

.gooseignore

Keep secrets, key material and vendored directories out of the agent's reach.

Secrets handling

Keyring by default. secrets.yaml is a plaintext fallback, so treat it as one in CI and containers.

`before you connect any mcp server`

Read what it does. An extension is code, and you are giving it your agent's privileges. The docs have a guide on judging whether an MCP server is safe.

Reference: goose-docs.ai/docs/guides/security

— 11 · MCP SURFACE & PLATFORM OPERATIONS

Protocol capabilities, and running at scale

MCP PROTOCOL CAPABILITIES

MCP Sampling
An MCP server can ask goose's model to think, which turns a passive tool into one that reasons.

MCP Elicitation
Extensions ask you for structured input mid-task rather than guessing or failing.

MCP Roots
How goose shares your working directory with roots-aware extensions.

MCP Apps / MCP-UI
An extension can render an interactive UI surface inside the chat.

RUNNING GOOSE AT SCALE

Multi-Model Config
Cheap model for routine turns, strong model for planning. Trades cost against results.

LLM Rate Limits
How to work within provider request ceilings without stalling a run.

Remote Server
Run goose serve on a VM and point Desktop at it. Heavy work leaves your laptop.

Run Tasks
goose run with a file or one-liner: headless execution for CI and cron.

Reference: goose-docs.ai/docs/category/guides · full index

— 11 · MCP SURFACE & PLATFORM OPERATIONS

And the rest of the index

Custom Distributions

Fork goose into your own branded distro with preconfigured providers and bundled extensions.

Logging & Usage Data

Unified local storage of conversations; OTLP export; telemetry is opt-in.

ALSO IN THE GUIDES

Enhanced Code Editing | File Management | Terminal Integration | Desktop Sidebar | Offline Docs | CLI Commands | Quick Tips | VMware Tanzu

ACP is to agents what LSP was to language tooling

One interface, many clients, many agents.

CLIENT

Zed · JetBrains · goose Desktop · your own app

← ACP →

stdio / JSON-RPC

AGENT

goose acp, the runtime, running as its own process or daemon

The agent then reaches its tools over **MCP**: extensions and tools, one layer further down.

Reference: agentclientprotocol.com · goose-docs.ai/docs/guides/acp-clients · /guides/acp-providers

It runs in both directions

1 · goose as an ACP server

`goose acp` starts goose as an ACP server over stdio. Editors that speak ACP, such as Zed and JetBrains, connect to it and drive the agent from inside the editor.

The interface and the runtime stay separate processes.

2 · ACP agents as goose providers

goose can also delegate to an external ACP agent such as Claude Code (`claude-acp`) or Codex (`codex-acp`). That agent runs the tools, while goose passes its configured extensions through as MCP servers.

You keep goose's sessions, recipes and scheduling on top of someone else's model subscription.

ACP separates the interface from the runtime. One protocol means an agent is not trapped inside the client that shipped it, and it is the protocol the goose Development Kit builds on.

Reference: agentclientprotocol.com · goose-docs.ai/docs/guides/acp-clients

Three components, one interactive loop

This is the whole system.

Interface

Desktop app or CLI. Collects your input, renders output, and starts agents, more than one at a time if needed.

Agent

The core logic that runs the interactive loop: calls the provider, executes tool calls, revises context, handles errors.

Extensions

MCP servers exposing tools. Built in, external, or ones you write. Capability comes from here.

The interactive loop

1
Human request
You state a goal.

2
Provider chat
Request plus available tool schemas go to the LLM.

3
Tool call
Model emits JSON; goose runs it and collects results.

4
Response to model
Results return. More tools needed? Repeat 2 to 4.

5
Context revision
goose prunes old or irrelevant content to manage tokens.

6
Model response
Final answer to you. The loop restarts on your reply.

repeat until done

Two properties that make it survivable

Errors do not break the flow

Invalid JSON, a missing tool, a failed command: goose catches these and returns them to the model as tool responses. The model then has what it needs to correct itself and carry on, instead of the run dying.

Context revision keeps the bill down

Summarise with smaller, faster models · drop stale content algorithmically · find-and-replace rather than rewriting whole files · ripgrep to skip system files · compress verbose command output.

Reference: goose-docs.ai/docs/goose-architecture/extensions-design

Not one agent app, but the infrastructure

THE THESIS

GDK is an open-source SDK in Rust for building agent applications that are model-agnostic, run locally, and do not depend on one provider. Instead of every team rebuilding the agent loop, tool calling and context management, they share the parts.

TWO WAYS TO INTEGRATE

ACP

Talk to goose as a separate process or daemon, keeping the runtime independent of your interface.

Rust API

Embed the agent in your application. goose Desktop and the goose CLI are built this way.

Building blocks on offer

- Agent orchestration
- Flexible agent loop
- In-process local models
- 50+ model providers
- Routing & model selection
- Context management
- Tools & MCP
- Memory
- Remote execution
- Automations & scheduling
- Skills
- Subagents
- Code mode
- Slash command infra

Docs	Source	Community	Recipes	Skills
goose-docs.ai	github.com/aaif-goose/goose	discord.gg/goose-oss	goose-docs.ai/recipes	goose-docs.ai/skills

— THANK YOU

Open standards over closed platforms.

Governed by the Agentic AI Foundation, under the Linux Foundation.

goose-docs.ai | github.com/aaif-goose/goose | discord.gg/goose-oss

Azeez Syed · AIYatra

Khaja Moinuddin Mohammad · Organiser and founder, AIYatra

Kubernetes over Koffee · AIYatra

COMMUNITY

Join AIYatra

An AI learning journey out of Manikonda, Hyderabad. Talks, workshops and hands-on sessions.

meetup.com/aiyatra

Scan to join